REFERENCE

# The smysl Document Format

*Every construct of the .smy surface syntax, with a worked example for each*

*Vladimir Ulogov · 2026 · smysl 1.8.0 · format smysl/1.0 · kernel smysl.kernel/0.1*

This reference covers one thing only: how to write a `.smy` document by hand — every record you can author, every field it takes, and the rules a validator will hold you to. For **why** the format is shaped this way, see `SMYSL_RATIONALE.typ`; this document only answers **how**. Every example below is real syntax, checked against the parser rather than invented for the page.

> **THIS GUIDE IS NOT THE CONTRACT**
>
> It is a **writer's** reference — descriptive, and free to explain more than it obliges. What another implementation must obey to interoperate is `SMYSL_FORMAT_SPEC.md`, which is normative and deliberately a fraction of this length: identity and how a uid is derived, the deterministic-CBOR constraints, record framing, the round-trip fixed point, rule X and the conformance classes. Nothing else is required.
>
> RFC SMYSL-1, which earlier versions of this guide referred to, is retired. It was the product idea rather than the doctrine, and the implementation outgrew it.

## How a document is put together

A `.smy` file is a flat sequence of **records**, each starting at column 0. Order does not matter to the meaning of the document — a claim can cite evidence that appears later in the file — though it is conventional to write things in the order a reader would want to meet them. Blank lines separate records but carry no meaning of their own.

### COMMENTS

A line beginning `#` or `//` at column 0 is a comment. Both markers, because an HJSON header inside a record already accepted both, and rejecting between records what was accepted within one made the surface contradict itself.

A comment is a comment **wherever it sits**, including inside a body. The alternative was worse: a body runs from the gist to the next record, so a comment between two records fell inside that range and became the previous unit's body, inventing content out of a note.

A body that genuinely needs to open a line with a marker escapes it — `\#` and `\//`, and `\\` for a line that already starts with a backslash. Only those three, and only at column 0, so prose full of Windows paths and LaTeX needs no thought. The writer puts the escape back, so `fmt` stays a fixed point.

New in 0.6. Before it there was no escape at all: a Markdown heading or a line of C++ in a body was read as a comment and dropped in silence, which is a poor way to treat exactly the content this format carries.

No record carries a comment, so canonical form cannot reproduce one and `fmt` warns before dropping them. A note that has to survive belongs in a unit — typically `@question` or `@prose` — where it is content and travels like content.

Inside a header the markers work the same way, which has one consequence for values: a value that **begins** with `#` or `//` must be quoted, or the rest of the line — closing brace included — is read as a comment. The writer quotes such a value for you. A marker in the **middle** of a value is ordinary text, because a quoteless value runs to `,`, `}`, `]` or end of line without stopping at either marker. So `ref: grafana://board/12` needs no quotes and gets none.

New in 0.2. Before that there was no comment syntax at all, and such a line was `SMY-E001`. The quoting rule for values is 0.4: until then the writer emitted such a value bare and lost the record it belonged to.

Any other line that is not part of a record's header, gist, body, or detail is a diagnostic (`SMY-E001`), not an ignored remark.

Four kinds of record are ever hand-authored in surface syntax:

| CONSTRUCT | WHAT IT DECLARES |
|---|---|
| `@doc` | The document header — at most one per file, and optional. |
| a unit | `@claim`, `@evidence`, and the other thirteen kernel types — the thing being claimed, cited, or asked. |
| `@rel` | A typed edge between two units — `causes`, `rebuts`, `warrant`, and eleven more. |
| `@thread` | A named, ordered, role-annotated walk over units — a brief, a narrative, an analysis. |
| `@schema` | A declaration of an extension: the unit types and relation kinds it adds. Rarely needed, and one line when it is. |
| `@withdraw` | An edge that should no longer be followed, and who withdrew it. Since 1.4. |
| `@resolve` | A record that a disagreement was reviewed, and by whom. Since 1.4. |

Other record kinds exist in the format and none is something you write directly. An **attestation** is stamped on automatically by whatever tool ingests or authors a unit; a **contention** is what `merge`

produces when two documents disagree; a **pack manifest** is `pack`'s receipt; a **schema declaration** names an extension a store depends on.

A **label binding** is the one worth knowing about, because it explains a number. Writing `@claim c/cache-cold` declares both a unit and a name for it, and the name travels as its own record — so a labelled unit yields two. That is why `check` reports more records than you typed, and why `units` rather than `records` is the figure to read when you want to know how much document you have.

The binding has to be separate because a label is **not identity**: it is not hashed, and renaming one must not produce a different unit. New in 0.2 — before it, labels survived a parse and not a store round trip, so a document that had been through `merge` came back with every reference spelled as a bare uid.

## The `@doc` header

If present, `@doc` must be the first thing in the file, and it fixes the document's own identity and the terms under which everything else in it should be read:

```
@doc smysl/0.1 {
  id: v/f1
  intent: incident-brief
  lang: en
  requires: ["smysl.kernel/0.1"]
  granularity: { profile: default }
  roots: [f/root-cause]
}
```

| FIELD | DEFAULT | MEANING |
|---|---|---|
| `id` | `v/doc` | This document's own identifier — a view id, `v/<slug>`. |
| `intent` | empty | A free-text label for what the document is for, e.g. `incident-brief`. |
| `lang` | `en` | A BCP-47 language tag — up to 35 characters, hyphen-separated alphanumeric segments. |
| `requires` | empty | Schemas a full-fidelity reader must implement: the kernel schema itself and any extensions. |
| `roots` | empty | The units this view is **about** — everything reachable from them is in the view; nothing is copied. |
| `threads` | empty | Thread ids this view publishes, if any. |
| `granularity` | `default` profile | How much one unit is allowed to say — see below. |

A view is not a container: `roots` names starting points, and everything reachable from them via `deps`, `grounds`, and relation edges belongs to the view at zero copying cost. That is also what makes merging two documents a plain union rather than a conflict.

### Granularity

`granularity` bounds how much a single unit is allowed to say, as three presets or a custom mix. Granularity constrains **production** — what you should write — not the store: a merged corpus mixing granularities is legal, not an error.

| PROFILE | GIST (L0) | BODY (L1) | ADMISSION |
|---|---|---|---|
| `coarse` | ≤ 30 tokens | 120 – 400 tokens | `topical` — one unit may bundle a topic |
| `default` | ≤ 30 tokens | 40 – 120 tokens | `single-assertion` — one unit, one claim |
| `fine` | ≤ 30 tokens | 20 – 60 tokens | `single-assertion` |

Under `single-assertion` admission — the default — a body that bundles more than one claim into a single unit is `SMY-E040`, an error, not a style complaint: split it into two units and a relation between them instead. A body outside its profile's range is only `SMY-W041`, a warning.

## Units — the shared anatomy

Every unit, whatever its type, shares the same header and the same three-level text. Here is the fullest ordinary example the format has:

```
@claim c/pool-saturation { status: inferred, grounds: [e/pool-wait], salience:
0.8 }
~ The eu-west connection pool is saturated.
  Waits climbed from a steady 2 ms baseline to over 300 ms in the same hour the 4.2
  rollout reached eu-west, and no other shard's pool moved.

--
Per-shard breakdown at hourly resolution: eu-west crosses 250 ms at 09:14 UTC,
forty
minutes after the rollout's first canary step there. No other shard's pool exceeds
its usual 5 ms band over the same seven-day window.
```

| FIELD | REQUIRED WHEN | WHAT IT IS |
|---|---|---|
| label | optional | The word right after the type, before `{`, e.g. `c/pool-saturation` — how the rest of the file refers to this unit. Not identity: two labels never make two units, and a unit needs no label at all if nothing else in the file must reference it by name. |
| `status` | always — defaults to `speculative` | How sure this unit is entitled to be — the six-rung ladder, next section. |
| gist (`~ …`) | always | The one required line of text — L0. Must sit on the line immediately after the header, with no blank line between. |
| body | needed by `derived` / `inferred` gists that lean on it | L1 — a paragraph or two, the ordinary prose account. |
| detail | only after a body | L2 — unbounded, introduced by a lone `--` line. A detail with no body above it is `SMY-E023`. |
| `deps` | when the gist needs context to parse | Other units this one's **wording** depends on to be understood — not evidence, just prerequisite reading. |

| FIELD | REQUIRED WHEN | WHAT IT IS |
|---|---|---|
| `grounds` | required by `derived` / `inferred` | Other units this one's **truth** rests on. Retract one and this unit's status is re-opened. |
| `source` | required by `measured` / `cited` | Where this unit grounds out externally — see below. |
| `salience` | optional | An authored override in `[0,1]`, clamped and quantised to 1/1024. Left unset, salience is computed rather than declared. |

The gist must immediately follow the header — not even a blank line may separate them (`SMY-E021`). A gist continuation line is indented by exactly two spaces; a body or detail line is not indented at all. The assembled gist is trimmed: whitespace around it is never content, because the reader strips the `~` sigil and the space after it, so a gist that carried a leading space could not be written back and read again unchanged.

> **WHAT IS NOT CONTENT**
>
> Identity is content, so anything the format treats as **not** content has to be settled before a uid is computed — otherwise two documents that say the same thing hash differently. Three things are normalised on the way in:
>
> - **Line endings.** A CRLF file and an LF file are the same document. Every carriage return at the end of a line is stripped, not just one. A carriage return **inside** a line is ordinary text and survives.
> - **Whitespace around a gist**, as above.
> - **Unicode form.** All text is normalised to NFC — including the unknown header keys and values that rule X carries verbatim. `é` written as one code point and `é` written as `e` plus a combining accent are the same text and get the same uid.
> - **A repeated field.** Writing `deps` twice in one header keeps the first and discards the rest. Before 0.5 the second was carried as an unknown key under rule X and written back out as a plain `deps:` line, which the next parse then read as the field — so the document lost content by being reformatted. There is no way to spell a second `deps` in surface syntax, so first-wins is the only rule that round-trips.
>
> Each of these was a real defect before 0.4, and the Unicode one was the worst of them: it failed silently in release builds, so two peers could write identical content and disagree about its identity.

### Status: the six-rung ladder

| STATUS | NEEDS | MEANING |
|---|---|---|
| `unfounded` | — | Reachable only by retraction; MUST NOT be authored (`SMY-E034`). |
| `speculative` | neither | Offered, not yet grounded — the default if you omit `status`. |
| `inferred` | `grounds` | A model reasoned it out from other units. |
| `derived` | `grounds` | Computed from other units by a deterministic procedure. |
| `cited` | `source` | A source you can open says so. |
| `measured` | `source` | An instrument recorded it. |

Two mechanical rules sit on top of this table. **Rule M**: a unit's status can never outrank the weakest thing in its own `grounds` (`SMY-E030`) — a `derived` claim resting on a `speculative` one is itself no better than speculative, whatever status you write. **Rule T**: the **rung** an agent is working from caps what status it may even attempt:

| RUNG | CEILING | WHAT IT IS |
|---|---|---|
| `computed` | `derived` | A deterministic tool, calculation, or parser. |
| `document` | `cited` | A user-supplied document or dataset. |
| `web` | `cited` | Fetched content, gated. |
| `model` | `inferred` | The model's own parametric knowledge — never higher, however confidently phrased. |

No rung reaches `measured` on its own, and the way it **is** reached is worth stating exactly, because the obvious reading is wrong. It takes an attestation recording `op: imported` *at the* `computed` *rung* — a deterministic tool transcribing a reading. The op alone is not enough: `ingest` also records `imported`, because it too transcribes rather than authors, so unlocking on the op by itself would let a model mark anything it read in a document as measured. Ingest runs at `document`, `web` or `model` and stays capped there.

> **WHY YOU CAN STILL TYPE IT**
>
> You may write `status: measured` in a `.smy` file and `check` will pass it, which looks like a contradiction and is not. Rule T is checked against a unit's **attestation**, and a `.smy` file cannot express one — provenance has no surface syntax. With nothing recorded about where a unit came from, there is no ceiling to check it against, so the rule stands aside rather than guessing. What you have written is an unchecked claim, not a licensed one, and it becomes checkable the moment the unit acquires provenance.
>
> The producer that writes `measured` **with** the provenance that licenses it is the `import` command, which transcribes a table of readings. A unit whose attestation records some other op is `SMY-W035`.

### Keys the grammar does not know

A header key that is not one of the fields above is **not** an error. It is kept, verbatim, in the unit's payload and written back out in the same place — the surface half of rule X.

One word is not available for this any more. `lang` on a unit is a **core** key — the language the unit's own text is written in, which is part of what the text says and therefore part of its identity — so a payload key of that name has to be written quoted: `"lang": …`. Saying nothing means "the language of the view that presents this unit", which is what every unit meant before the key existed. `und` and `mul` are permitted values, and both are different from silence. A reader from a later version, or from a team that records something yours does not, loses nothing by passing through yours.

```
@data t/latency { status: measured, source: { kind: file, ref: "by-region.csv" },
                  x.stats/method: "empirical-quantile",
                  columns: ["region", "p50_ms", "p95_ms"], rows: 6 }
~ Checkout latency by region, June 2026.
```

`rows`, `columns` and `x.stats/method` are none of the kernel's business. They survive a parse, a round trip through the binary encoding, and a merge — and they come back in canonical order rather

than the order you typed them, which is why `smysl fmt` returns the unit above with `rows` first and `x.stats/method` last. Keys sort by encoded length, then content. That is not tidiness: identity is computed from the encoded bytes, so two people who record the same thing in a different order still compute the same unit.

> **TWO NAMES THE TOOLING WRITES**
>
> Ingest puts two of its own keys in the same place, and you will meet them reading a staged file rather than writing one. `ingest:quote` carries the span of the source document a unit was drawn from — checked against that document, so a quote that is not in it is an error rather than a decoration. `ingest:unrepaired` marks a span that survived its repair budget and was kept as opaque prose rather than dropped. Neither is reserved by the grammar; both are conventions you can read, write, or ignore.

### Source references

```
source: { kind: metric, ref: "pool.wait_ms{shard=eu-west}", captured: 2026-07-09 }
```

`kind` is one of five, `ref` (or the longer spelling `reference`) is a free-text pointer whose shape depends on the kind, and `captured` is an optional `YYYY-MM-DD` date.

Two more are optional. `observed` is the instant an observation was taken, in epoch milliseconds — a date is too coarse to order two readings a minute apart, which is the whole point of a time series. `published` is when the thing the unit came from was published, **as the source states it**, in EDTF: `1920`, `1920?` if the source hedges, `1920/1930` for a range, `201X` for a decade. The uncertainty belongs in the record, because the alternative is inventing `1920-01-01T00:00:00Z` and calling it provenance. The three answer different questions and a producer must not derive one from another.

Two more arrive with the library layer, and they come as a pair. `span` is `[start, end]` — byte offsets into a part's normalised bytes, half-open — and `manifest` is the `m3:` id of the manifestation those offsets are an offset **into**. A unit carrying a span must carry its manifest, because offsets with no manifestation name a place in bytes nobody can say they obtained the same way; the format refuses the pair's absence rather than dropping the span, which is inside the unit's identity.

```
source: { kind: doc, ref: "t3:...#Ps.14.1", span: [198, 252], manifest: m3:... }
```

The span is authoritative and the locator after `#` is informative. A locator is a reader's name for a place — a verse, a line, a JSON pointer — and when the two disagree the span wins and `SMY-W405` says so; a span reaching past the end of its part is `SMY-E404`. That ordering is the whole reason to carry both: the span is what the extraction read, and the locator is what a person can say out loud.

| KIND | TYPICAL `REF` | CAPTURE DATE |
|---|---|---|
| `url` | a web address | **Required** — a fetched page is unverifiable later without one. |
| `file` | a path or filename | optional |
| `metric` | a metric name or query | optional |
| `tool` | a tool or command name | optional |
| `doc` | a document reference or anchor | optional |

## The thirteen unit types

`KernelType` actually enumerates fifteen names, but two of them — `contention` and `packinfo` — name records that tooling produces (`merge`, `pack`); nothing below shows them being hand-authored, because in ordinary use they never are. The other thirteen are yours to write. Each example is complete and independently valid.

| TYPE | WHAT IT RECORDS |
|---|---|
| `claim` | An assertion about the world. |
| `evidence` | A reading, instrument, or observation offered to ground a claim. |
| `definition` | What a term means, fixed so later claims can rely on it. |
| `question` | An open prompt the corpus has not yet answered. |
| `hypothesis` | A candidate answer, offered before it is tested. |
| `finding` | A conclusion resting on other claims. |
| `procedure` | A course of action — a "what to do," not a "what is." |
| `decision` | A commitment a person or team made. |
| `constraint` | A rule the rest of the reasoning has to respect. |
| `observation` | A plain record of what was seen. |
| `data` | A pointer to a dataset, rather than one reading. |
| `artifact-ref` | A pointer to something outside the document — a dashboard, a repo. |
| `prose` | Free-standing text that fits none of the above — still a first-class, citable unit. |

```
@evidence e/pool-wait { status: measured, source: { kind: metric, ref:
"pool.wait_ms{shard=eu-west}", captured: 2026-07-09 } }
~ Pool acquisition wait rose from 2 ms to 310 ms over the same window.
```

An instrument reading grounds out externally, so `measured` demands a `source` — here a metric reference with the date it was captured.

```
@claim c/pool-saturation { status: inferred, grounds: [e/pool-wait] }
~ The eu-west connection pool is saturated.
```

A model reasoned this from the evidence above; `inferred` demands `grounds`, and rule M means this claim can never outrank `e/pool-wait` even if the wording sounds certain.

```
@definition d/p95 { status: cited, source: { kind: doc, ref: "sre-
handbook#latency" } }
~ The 95th percentile of request latency over a one-minute window.
```

A definition is grounded in a document a reader can open, not the author's own say-so — without it, "tripled" in a later claim would have no fixed meaning.

```
@question q/root-cause { status: speculative }
~ What actually caused the eu-west latency spike?
```

A question needs neither `source` nor `grounds` — `speculative` is the floor, an open prompt nothing has answered yet.

```
@hypothesis h/pool-exhaustion { status: speculative }
~ Connection-pool exhaustion under the 4.2 rollout is driving the p95 rise.
```

A candidate answer to the question above, offered before it is tested against evidence.

```
@finding f/root-cause { status: inferred, grounds: [c/pool-saturation, c/
regression] }
~ Pool saturation is the leading cause but is not consistent with the canary.
```

A finding rests on other claims rather than on raw evidence; rule M caps it at the weakest of the two grounds it names.

```
@procedure p/rollback { status: speculative }
~ Roll the eu-west pool config back to the 4.1 settings and re-run the canary.
```

A course of action, not a fact — still a checkable unit, just one whose content is a "what to do."

```
@decision de/rollback-approved { status: inferred, grounds: [f/root-cause] }
~ Approved: roll back the eu-west pool config tonight.
```

A commitment, grounded in whatever finding justified it — retract the finding and this decision's grounding is reopened along with it.

```
@constraint co/no-friday-deploys { status: cited, source: { kind: doc, ref: "ops-
runbook#change-freeze" } }
~ No production rollbacks after 16:00 on a Friday.
```

A rule the rest of the reasoning has to respect, sourced from wherever that rule is actually written down.

```
@observation ob/canary-quiet { status: measured, source: { kind: metric, ref:
"canary.p95" } }
~ The 4.2 canary showed no latency regression over the same window.
```

A plain record of what was seen — it doesn't ground anything by itself yet, but it can be cited later, exactly like any other unit.

```
@data da/latency-series { status: measured, source: { kind: file, ref: "auth-
p95-2026-07.csv" } }
~ Hourly p95 auth latency, all shards, 2026-07-02 through 2026-07-09.
```

Points at a dataset rather than asserting a single reading — the source is the file itself.

```
@artifact-ref ar/dashboard { status: cited, source: { kind: url, ref: "https://
grafana.internal/d/api-latency", captured: 2026-07-09 } }
~ The latency dashboard the on-call team watches.
```

Points at something that exists outside the document. Its source kind is `url`, the one kind where `captured` is mandatory rather than optional.

```
@prose pr/note { status: speculative }
~ Worth a follow-up: check whether the 4.1 pool size was ever tuned for eu-west's
traffic shape.
```

The escape hatch — a citable aside that doesn't fit any more specific type.

> **NAMESPACE CONVENTION, NOT GRAMMAR**
>
> `e/`, `c/`, `d/`, `f/`, and the rest above are a **habit**, not a rule. A label is any `ident/ident` pair — lowercase letters, digits, `-`, `_` — and nothing in the grammar reserves any prefix for any unit type. Pick short, memorable prefixes and stay consistent within a file; the parser does not care what they are.

## Relations (`@rel`)

A relation is a typed, directed edge between two units, written on one line:

```
<from> --<kind>--> <to> [{ weight: <0..1>, note: <ref> }]
```

```
@rel c/canary-clean --rebuts--> c/pool-saturation { weight: 0.6 }
```

A direct objection. `rebuts` is the one kind rule R pins: wherever `c/pool-saturation` survives into a packed or rendered view, this edge — and the unit it points from — travels with it. `weight` is an optional strength for the rebuttal itself, clamped to `[0,1]`.

```
@rel c/pool-saturation --causes--> c/regression
```

Says the saturation claim is what produced the regression. `causes` is one of three **ordering** kinds (with `enables` and `sequences`) that a thread's automatic ordering can sort by, and one of two **support-bearing** kinds (with `answers`) that carry salience rank forward through the graph.

```
@rel d/p95 --warrant--> c/regression
```

Says the definition licenses reading the claim the way it is read — without it, "tripled" has no fixed meaning to check.

```
@rel c/pool-saturation-v2 --supersedes--> c/pool-saturation
```

Marks the newer claim as replacing the older one outright. `supersedes` and `retracts` are the two **lifecycle** kinds — the only ones that change what the corpus should currently be read as believing, rather than adding a new relationship alongside what was already there.

```
@rel c/pool-saturation --x.sre/mitigates--> p/rollback
```

An unrecognised kind — anything shaped `x.<domain>/<kind>` — is kept and stays routable rather than dropped (`SMY-W013`); a reader without the `x.sre` extension treats it as a plain `elaborates` edge instead of losing it.

The warning goes away when the file declares the kind it uses:

```
@schema x.sre/v1 { version: 1, relations: [x.sre/mitigates] }
```

The declaration travels with the store through `merge` and `fmt`. A misspelled key in it is an error rather than something skipped — a `relation:` quietly ignored would leave the kind warning under a file that appears to declare it — and declaring a kernel kind such as `causes` is `SMY-E012`, because an extension may add but never redefine.

An edge can be **withdrawn** when a reviewer finds the edge itself wrong — two statements a model matched as opposed that are not — without retracting either unit:

```
@withdraw c/canary-clean --rebuts--> c/pool-saturation { agent: human:vladimir, ts:
[1726500000000, 0], reason: c/not-opposed }
```

The `@rel` line stays and the edge is no longer followed: a withdrawn rebuttal no longer travels with its claim. `retracts` and `supersedes` edges cannot be withdrawn. When a disagreement is real and both sides stand, the review is recorded instead, naming the edge or the contention merge reported:

```
@resolve k/ccm3actwjjti65famnoe6mapo5d { agent: human:vladimir, ts: [1726500000000,
0], note: c/both-stand }
```

A resolution decides nothing — the rebuttal still travels with the claim — and takes the item off the list `smysl review` prints. `ts` is milliseconds and a counter, as for `@thread`. Both words are reserved since 1.4, so an older `smysl` rejects a file that uses them.

The fourteen kernel kinds, for reference:

| KIND | READS AS |
|---|---|
| `elaborates` | adds detail without changing the claim |
| `contrasts` | sets two things against each other |
| `concedes` | grants a point while maintaining the larger claim |
| `causes` | one thing produced another **(ordering, support-bearing)** |
| `enables` | one thing made another possible without directly producing it **(ordering)** |
| `exemplifies` | gives a concrete instance of a general claim |
| `conditions` | states an “only if” dependency |
| `sequences` | orders two things in time, with no causal claim **(ordering)** |
| `answers` | resolves a question **(support-bearing)** |
| `rebuts` | directly objects to the target — pinned by rule R |
| `warrant` | licenses an inference or a reading |
| `backs` | supports a warrant |
| `supersedes` | replaces the target outright **(lifecycle)** |
| `retracts` | withdraws the target outright **(lifecycle)** |

## Threads (`@thread`)

A thread is a named, ordered, role-annotated walk over units — the same units the corpus already has, arranged into a reading order for one audience. Its header takes a `schema` (which fixes the allowed roles), an `owner` (an agent id — a thread is **owned** state, last- writer-wins **per owner**, so two agents publishing the same thread id never conflict), and a gist. Steps follow, indented by two spaces:

```
  <role> → <unit>[: <note>]
```

Each of the five schemas fixes a different sequence of roles. One worked example per schema:

```
@thread t/brief { schema: brief, owner: "human:vladimir" }
~ Auth p95 tripled in eu-west; pool saturation is leading but contested.
  bottom-line → f/root-cause
  support → c/pool-saturation
  risk → c/canary-clean
```

`brief` — `bottom-line, support, risk, ask` — walks a reader from the one-line verdict down to what backs it and what argues against it. Not every role has to be used.

```
@thread t/root-cause-qa { schema: qa, owner: "human:vladimir" }
~ What caused the latency spike, and how sure are we?
  question → q/root-cause
  evidence → e/pool-wait
  answer → f/root-cause
  caveat → c/canary-clean
```

`qa` — `question, evidence, answer, caveat` — is literally a question answered in public, with its evidence and its own best objection attached.

```
@thread t/incident-story { schema: narrative, owner: "human:vladimir" }
~ How a quiet Tuesday became an eu-west incident.
  setup → ob/canary-quiet
  complication → e/pool-wait
  turn → c/pool-saturation
  resolution → de/rollback-approved
  coda → f/root-cause
```

`narrative` — `setup, complication, turn, resolution, coda` — orders the same kind of units as a story, for an audience that needs to follow how the conclusion was reached.

```
@thread t/deep-dive { schema: analysis, owner: "model:anthropic/claude-opus-5" }
~ Full trace of the eu-west pool-saturation analysis.
  context → ob/canary-quiet
  tension → e/pool-wait
  approach → h/pool-exhaustion
  finding → c/pool-saturation
  rebuttal → c/canary-clean
  implication → f/root-cause
  next → p/rollback
```

`analysis` — `context, tension, approach, finding, rebuttal, implication, next` — is the longest schema, a full research trace with its own slot for what to do next.

```
@thread t/rollback-plan { schema: plan, owner: "human:vladimir" }
~ Rolling eu-west back to 4.1 tonight.
  goal → de/rollback-approved
  constraint → co/no-friday-deploys
  step → p/rollback
  risk → c/canary-clean
```

`plan` — `goal, constraint, step, decision, risk` — orders the units that make up a course of action rather than an argument.

> **ARROWS AND NOTES**
>
> Either `→` (U+2192) or the ASCII `->` is accepted when **reading**; the canonical writer (`smysl fmt`) always emits `→`. A step may carry a trailing `: note` after the reference, as free text — `support → c/pool-saturation: the strongest single reading`.

`risk` appears in both `brief` and `plan` — it is one role shared between schemas, not two separate ones. A step whose role the thread's schema does not declare is not rejected; it is reported as a **foreign role**, kept rather than dropped.

## Identifiers, at a glance

| KIND | SHAPE | EXAMPLE |
|---|---|---|
| Label | `ident/ident`, lowercase | `c/pool-saturation` |
| Thread / view / contention id | same shape, label-typed | `t/brief`, `v/f1`, `k/pool-vs-index` |
| Agent id | `(model\|human\|tool):name[/tag]` | `model:anthropic/claude-opus-5` `human:vladimir` |
| Kernel type | a bare word | `claim`, `evidence` |
| Kernel schema | `smysl.kernel/MAJOR[.MINOR]` | `smysl.kernel/0.1` |
| Extension schema | `x.<domain>/<segment>` | `x.sre/incident` |
| Language tag | BCP-47-shaped, ≤ 35 chars | `en`, `en-GB`, `zh-Hans-CN` |
| Content hash (uid) | `b3:` + base32, machine-facing | `b3:cvhirtgs2mpvli2ethhyeo32uf…` |

You will essentially never type a `b3:` hash by hand — that is exactly what labels are for. A hash is what a unit's identity actually is; a label is the name you gave it so you never have to write the hash yourself. An agent id's kind is fixed to one of three words; a model id's segment after the first `/` groups corroboration by provider, which is why `model:anthropic/claude-opus-5` and `model:anthropic/claude-haiku-4-5` still corroborate as the same provider.

## Validation, at a glance

A parse never simply fails: a malformed record becomes a diagnostic with a byte span, and parsing recovers at the next record start, so one bad unit costs you that unit, not the whole file. The codes below are the ones ordinary hand-authoring runs into.

| CODE | WHAT IT MEANS |
|---|---|
| `SMY-E001` | General surface parse error — a malformed header, an unknown type, a stray line. |
| `SMY-E020` | Body or detail names a unit's identifier that isn't listed in `deps` or `grounds`. |
| `SMY-E021` | Missing gist, or the gist doesn't immediately follow the header. |
| `SMY-E022` | Gist exceeds the profile's `l0_max`. |
| `SMY-E023` | `detail` present without a `body` above it. |
| `SMY-E030` | Rule M — status exceeds the weakest of its own grounds. |
| `SMY-E031` | `derived` / `inferred` with empty `grounds`. |
| `SMY-E032` | `measured` / `cited` without a `source`. |
| `SMY-E033` | Rule T — status exceeds the ceiling for the authoring rung. |
| `SMY-E034` | `unfounded` authored directly — only reachable by retraction. |
| `SMY-E040` | More than one assertion in a body under `single-assertion` admission. |
| `SMY-E060` | A label referenced but never defined anywhere in the document. |
| `SMY-E061` | A cycle in `deps` or `grounds`. |
| `SMY-W013` | Unknown relation kind — kept, treated as `elaborates` for closure. |
| `SMY-W035` | `measured` on an authored (not imported) unit. |
| `SMY-W041` | Body length outside the profile's `l1_range` — advisory only. |

Three more you will only meet in a document a model produced, never one you typed. They are listed because you will read them in a staged file and should know they are not your mistake:

| CODE | WHAT IT MEANS |
|---|---|
| `SMY-W036` | Rule M applied at ingest — a unit claimed more than its grounds support and was lowered to what they do. The record of a model overreaching, not work outstanding. |
| `SMY-E307` | An attributed `ingest:quote` does not occur in the source document. A fabricated attribution, and the one thing here worth another turn of the model. |
| `SMY-W308` | The quote occurs only loosely — elided or reworded. Honest attribution with a clause dropped. |

### AT A GLANCE

- Four constructs are hand-authored: `@doc` once, units as many as you need, `@rel` for typed edges, `@thread` to walk them in an order.
- Every unit shares one anatomy — label, status, gist/body/detail, `deps` / `grounds`, `source`, `salience` — whatever its type.
- Status only ever falls, never rises, across `grounds` (rule M) and is ceilinged by the rung doing the authoring (rule T); `measured` needs `op: imported` at the `computed` rung, which in practice means `smysl import` and nothing else.
- Thirteen unit types are yours to write; `contention` and `packinfo` are tooling-only.
- Fourteen relation kinds, five thread schemas, twenty-four roles — all closed sets, all with an escape hatch (`x.<domain>/<kind>`) that degrades gracefully rather than being dropped.
- Header keys the grammar does not know are kept verbatim in the payload rather than refused, so a document from a later version or another team passes through yours intact.
- A `#` or `//` line at column 0 is a comment, and no record carries one — `fmt` warns before dropping them. There is still no blank line permitted between a header and its gist, and no reserved label namespaces: only what the grammar actually checks.
- A labelled unit yields two records, because the label travels as its own binding rather than inside the hashed content that decides identity.